AI651 – Deep Learning for Space, Time and Graphs
Assignment 2

Saad Ali [25280011]

Fall 2026

GitHub repository: `https://github.com/saadali18/deep-learning-for-space-time-and-graph/tree/main/PA2`

Question 2 evidence: all numbers come from the scripts in `PA2/Question2`, run on an Apple M3 Pro (18 GB unified memory) with PyTorch 2.10 on the MPS GPU backend. Learned models are reported as mean (seed 0, seed 1). Unless stated otherwise, results are on the 1,000 test videos.

Question 1 – River Forecasting Proposal

We propose a **graph-structured space–time forecaster**: a small neural network whose spatial connections follow the river (reservoir → S1 → S2 → S3 → S4, with tributaries joining at the right places). Each source is first aligned to one 10-minute grid, then a temporal block reads each location's recent history, and spatial blocks move information downstream with travel times that are learned and can change with the state of the river. Every ten minutes the system outputs, for each of S1–S4, the median water level one hour ahead and an 80% interval. The physics we are confident about (which way water flows, where tributaries join, that travel time grows with distance and shrinks at high flow) is built into the structure. What we are not confident about (exact lags, rainfall–runoff response, the unmonitored tributary) is learned from the historical record.

1. Architecture diagram

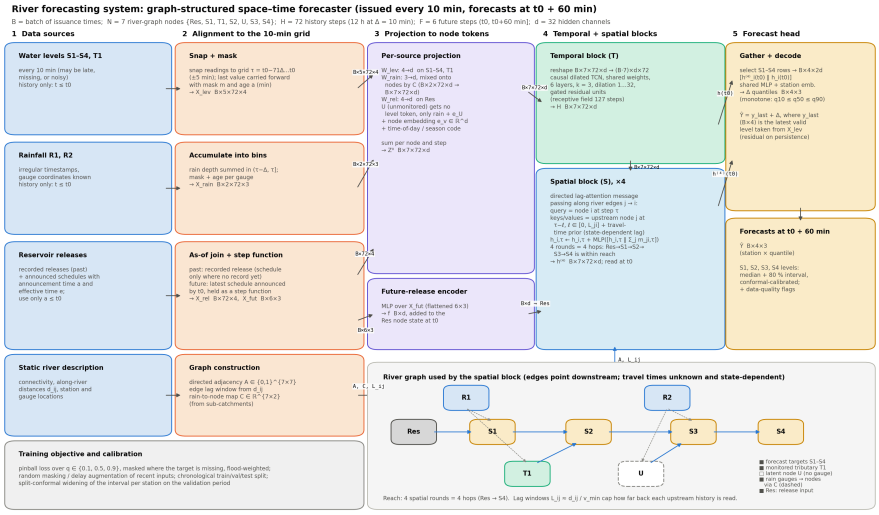


Figure Q1: Data flow from the four data sources (column 1) through alignment to the 10-minute grid (2), projection of every source to node tokens of width d (3), a temporal block and four spatial message-passing rounds on the river graph (4), to quantile forecasts for S1–S4 (5). The tensor shapes on the arrows include the batch axis B (independent issuance times). Bottom right: the 7-node river graph used by the spatial block. U is a latent node for the unmonitored tributary. It has no level gauge and receives only rainfall.

2. Assumptions

Supplied facts (taken as given, not assumed): F1 levels at S1–S4 and T1 every 10 min (historical); F2 rainfall at R1 and R2 with timestamps and gauge locations; F3 recorded reservoir releases, plus future release schedules with their announcement and effective times; F4 station locations, connectivity and along-river distances; F5 a monitored tributary (T1) joins between S1 and S2, an unmonitored one between S2 and S3; F6 precise travel times and the unmonitored inflow are not supplied, and travel times change with conditions; F7 future rainfall and future levels are unavailable at issuance.

#	Assumption	Why it is reasonable	Effect on the design
A1	At least 3–5 years of concurrent 10-min records, including several high-flow events.	Authorities that run gauges usually archive several years. Fewer than a handful of floods would make any learned model unreliable at the extremes.	Allows a model of roughly 10^5 parameters, a chronological train/validation/test split, and flood-weighted training. With a shorter record we would shrink the model and lean on the persistence-plus-routing fallback.

#	Assumption	Why it is reasonable	Effect on the design
A2	All timestamps share one clock (UTC); level readings are nominally on 10-min marks with at most ±5 min jitter.	Standard telemetry practice.	Readings are snapped to the nearest grid point. A reading outside ±5 min is treated as missing at that point rather than shifted further.
A3	Reporting delays are usually under 10 min, but any gauge can occasionally be late by up to a few hours. Gauges report independently.	Typical of radio or cellular telemetry.	Each input carries a mask and an "age" feature. Training randomly delays recent inputs (Section 5).
A4	Missing data are mostly short gaps (a few % of points) plus rare multi-hour outages. Bad values can be found by range, spike and flat-line checks.	Common failure modes of field sensors.	A quality-control step sets mask = 0 for flagged values. Masking augmentation teaches the model to fall back on neighbouring gauges.
A5	Level sensors have about 1 cm noise and a fixed datum. Any datum or rating change is documented.	Required for any level-based forecast to be meaningful over years.	Levels are normalised per station with fixed statistics. A documented datum change is corrected before training.
A6	Rain gauges report depth (tipping bucket or interval totals). R1 is representative of the upper catchment (local inflow to S1 and the T1 basin) and R2 of the unmonitored tributary and the area draining to S3.	Inferred from the gauge locations in the project setting.	Rain is summed into 10-min bins and mapped to nodes by a matrix C (7×2). C is initialised from this sub-catchment guess and then learned.
A7	Releases are recorded as discharge (m^3/s). Schedules are usually honoured, but actual releases may differ in timing or amount (gate operations, emergency spills).	Operational reality at dams.	The model is trained on the schedule *as known at issuance* and also sees recent schedule-versus-record deviation (Section 5 iv).
A8	**Dynamics:** influence is downstream-dominant. Over one hour, backwater effects from downstream are small compared with upstream and local inputs.	The reach is a steep valley river (as illustrated). Flow is gravity-driven, so disturbances propagate mainly downstream.	Graph edges point downstream only. A station's own history still captures any slow local backwater effect.
A9	**Dynamics:** travel time between neighbouring nodes is positive and bounded. It lies roughly between $d/v_{\max}$ and $d/v_{\min}$ with wave celerity $v \in [0.3, 3]$ m/s, and it *decreases* as flow rises (kinematic-wave behaviour).	Standard open-channel hydraulics. Celerity grows with depth.	Each edge reads only a bounded window of upstream lags, $L_{ji} = \lceil d_{ji}/(v_{\min}\Delta) \rceil$ steps (capped at H). The lag weights depend on the current state, so the model can shorten the lag at high flow.
A10	**Dynamics:** over a one-hour horizon, levels are driven mainly by water that is already in the system: upstream flow, the release schedule, and rain that has *already* fallen. Rain falling after t_0 barely affects main-river levels within the hour.	Rainfall-to-runoff lags for catchments of this size are typically longer than one hour.	No future-rain input is used. Rain falling now on small local areas is a residual risk, handled by wider intervals (Section 6).
A11	The unmonitored tributary's inflow is correlated with recent R2 (and R1) rainfall. Its effect shows up first at S3 and then at S4. There are no other significant ungauged inflows or abstractions.	It drains the catchment where R2 sits.	A latent node U, fed only by rainfall, connects to S3. S3's own history captures how much the tributary is currently contributing.
A12	Channel geometry, gauge ratings and reservoir operating policy are roughly stable over the record. Seasonal effects such as snowmelt in the mountainous headwaters are slowly varying.	Stability is needed for any data-driven model to generalise. The setting is alpine.	Inputs include a season code. The model is retrained periodically (e.g. quarterly) and monitored for drift.
A13	Data arrive in an operational database within minutes. A standard CPU server is available.	Normal for a monitoring authority.	The model is a fully specified forward pass with no online optimisation (feasibility in Section 5).
A14	Each station has a flood-warning threshold (or one will be supplied).	Flood preparedness is the stated purpose.	Used to up-weight high-water samples in the loss and for event-based evaluation.

3. Forecasting task and data specification

- **Issuance time** t_0: every 10-min mark (hh:00, hh:10, ...). The forecast uses every value *timestamped* $\le t_0$ that has *arrived* by t_0 + 2 min (processing allowance). For announced releases, only announcements made at $a \le t_0$ are used.
- **Grid**: Δ = 10 min. **History window**: H = 72 steps, i.e. the 12 h from $t_0 - 710$ min to t_0 (inclusive). Twelve hours is comfortably longer than any plausible reservoir-to-S4 travel time plus the rainfall–runoff lag (A9, A10), so every cause of the next hour's level falls inside the window. The temporal block's receptive field (127 steps) covers the whole window.
- **Horizon**: one target time t_0 + 60 min (6 steps ahead), predicted *directly* rather than by feeding forecasts back, so errors do not compound. **Outputs**: for each station $i \in$ {S1, S2, S3, S4} the quantiles $q \in \{0.1, 0.5, 0.9\}$ of $y_i(t_0 + 60 \text{ min})$. The median is the point forecast, and $[q_{0.1}, q_{0.9}]$ is an 80% interval.

Tensor	Dimensions	Axis meanings	Time coverage / timestamps
Level inputs X_{lev}	$B\times5\times72\times4$	batch (issuance times); gauge (S1, S2, S3, S4, T1); grid step; feature [normalised level, mask, age (min since the reading), 10-min change]	$\tau = t_0-71\Delta, \ldots, t_0$; readings snapped to marks within ±5 min
Rain inputs X_{rain}	$B\times2\times72\times3$	batch; gauge (R1, R2); grid step; [depth in bin, mask, age]	bin τ covers $(\tau-\Delta, \tau]$; from native timestamps $\le t_0$
Release history X_{rel}	$B\times72\times4$	batch; grid step; [release in effect at τ, is-recorded flag (vs. scheduled), recorded − scheduled deviation, mask]	$\tau = t_0-71\Delta, \ldots, t_0$; step function of effective times
Announced future release X_{fut}	$B\times6\times3$	batch; future step; [scheduled release, has-schedule flag, minutes since announcement]	$t_0+\Delta, \ldots, t_0+6\Delta$; only announcements with $a \le t_0$
Calendar code p	$B\times72\times4$	batch; grid step; [sin, cos] of time of day and of day of year	same grid as the history
Static graph	A: 7×7, d: 7×7, C: 7×2	node × node adjacency (downstream edges) and distances; node × rain gauge mapping	static
Latest valid level y^{last}	$B\times4$	batch; station S1–S4	most recent unmasked reading $\le t_0$ (its age is in X_{lev})
Output $\hat{Y}$	$\boldsymbol{B\times4\times3}$	**batch; station S1–S4; quantile (0.1, 0.5, 0.9)**	**valid at t_0 + 60 min**
Target Y, mask M	$\boldsymbol{B\times4}$, $\boldsymbol{B\times4}$	**batch; station**	**observed level at t_0 + 60 min (training/evaluation only)**

Alignment across locations and sampling times. All sources are placed on one common grid of $H = 72$ time steps ending at t_0, so step τ means the same physical time at every node. Levels are point samples and are *snapped* to the grid. Rainfall arrives at irregular times and is *accumulated* into each bin $(\tau-\Delta, \tau]$, which conserves rain depth. Releases are piecewise constant in time, so the release *in effect* at τ is read off a step function. Space is aligned through the graph rather than by interpolation. Each source is attached to a node (gauges to their own node, releases to Res, rain to nodes through C). Different feature sizes are made comparable by a per-source linear projection to width d. A source that has no observation at a node (for example U has no level) simply contributes nothing there.

Information available at issuance. Available: past levels at S1–S4 and T1, past rain at R1 and R2, past recorded releases, schedules announced by t_0 (including effective times after t_0), the static river description, and the calendar. Not available: any future level or rain, inflow from the unmonitored tributary, actual future releases, and readings that are still in transit. Training samples are rebuilt *as of* each historical t_0 (using announcement times and, where logged, arrival times), so the model never learns from information it would not have in operation.

4. Model computations and design justification

Notation. Nodes $\mathcal{V} = \{\text{Res, S1, T1, S2, U, S3, S4}\}$. Downstream edges $\mathcal{E} = \{\text{Res}\to\text{S1, S1}\to\text{S2, T1}\to\text{S2, S2}\to\text{S3, U}\to\text{S3, S3}\to\text{S4}\}$. $\mathcal{N}(i)$ is the set of upstream neighbours of i. $x_{v,\tau}$ is the aligned feature vector of node v at step τ, $d = 32$, σ is the logistic function, and $\|$ denotes concatenation.

(a) Projection to node tokens (diagram column 3):

$$z^0_{v,\tau} = W_{\text{src}(v)}\, x_{v,\tau} + \sum_{r\in\{R1,R2\}} C_{vr}\, W_{\text{rain}}\, x^{\text{rain}}_{r,\tau} + e_v + W_p\, p_\tau, \qquad z^0 \in \mathbb{R}^{B\times7\times72\times d}.$$

$W_{\text{src}(v)}$ is W_{lev} for S1–S4 and T1, W_{rel} for Res, and absent for U. e_v is a learned node embedding.

(b) Temporal block (T): a causal dilated temporal convolution network with gated residual units. Weights are shared across nodes (the node identity enters through e_v). For layers $l = 0, \ldots, 5$ with dilation $\delta_l = 2^l$:

$$u^l_{v,\tau} = \sum_{k=0}^{2} W^l_k\, z^l_{v,\tau-k\delta_l}, \qquad z^{l+1}_{v,\tau} = z^l_{v,\tau} + W^l_o\big[\tanh(u^{l,f}_{v,\tau}) \odot \sigma(u^{l,g}_{v,\tau})\big],$$

which gives $h_{v,\tau} = z^6_{v,\tau}$. The block is causal (it never reads $\tau' > \tau$) and has a receptive field of $1 + 2\cdot63 = 127$ steps $\ge H$. It captures each node's own dynamics: recession after a peak, rain accumulation, and the release pattern.

(c) Spatial block (S): lag-attention message passing, repeated for rounds $r = 0, \ldots, 3$, with $h^{(0)} = h$. Along each edge $j\to i$, node i at time τ attends over upstream node j's states at lags $\ell = 0, \ldots, L_{ji}$:

$$\alpha^{(r)}_{ji,\tau}(\ell) = \underset{\ell \le L_{ji}}{\mathrm{softmax}}\Big(\tfrac{1}{\sqrt{d}} \, (W_Q h^{(r)}_{i,\tau})^\top W_K \big[h^{(r)}_{j,\tau-\ell} \,\|\, \phi(\ell) \big] \; - \; \tfrac{(\ell \Delta - \bar{t}_{ji})^2}{2\sigma^2_{ji}} \Big), \qquad m^{(r)}_{ji,\tau} = \sum_{\ell} \alpha^{(r)}_{ji,\tau}(\ell) \, W_V h^{(r)}_{j,\tau-\ell},$$

$$h^{(r+1)}_{i,\tau} = h^{(r)}_{i,\tau} + \mathrm{MLP}\Big(\big[h^{(r)}_{i,\tau} \,\|\, \textstyle\sum_{j \in \mathcal{N}(i)} m^{(r)}_{ji,\tau} \big] \Big).$$

Here $\varphi(\ell)$ is a lag encoding. The second term is a **travel-time prior** centred on $\bar{t}_{ji} = d_{ji}/\bar{v}$, with a learnable typical celerity $\bar{v}$ and width s_{ji}, and L_{ji} is the hard cap from A9. Because the query includes node i's current state (its level), the attention can move its weight to shorter lags when the river is high. This is how varying travel times are handled. Before round 0, the future-release embedding $f = \mathrm{MLP}(\mathrm{vec}\, X_{\mathrm{fut}})$ is added to h_{Res,t_0}.

Reach. Each round moves information one hop downstream and up to L_{ji} steps back in time. Four rounds cover the longest path Res→S1→S2→S3→S4. So every gauge, rain gauge and release can influence every downstream forecast, and the temporal block lets each node see all 72 steps.

(d) Forecast mapping. For station $i \in \{\mathrm{S1}, ..., \mathrm{S4}\}$:

$$[\mu_i, g_i^-, g_i^+] = \mathrm{MLP}_{\mathrm{out}}\big([h^{(4)}_{i,t_0} \,\|\, h_{i,t_0} \,\|\, e_i] \big), \qquad \hat{y}_{i,0.5} = y_i^{\mathrm{last}} + \sigma_i \mu_i,$$

$$\hat{y}_{i,0.1} = \hat{y}_{i,0.5} - \sigma_i \, \mathrm{softplus}(g_i^-), \qquad \hat{y}_{i,0.9} = \hat{y}_{i,0.5} + \sigma_i \, \mathrm{softplus}(g_i^+).$$

The model predicts a *change* from the latest valid level (σ_i is the station's level scale). With all weights at zero this reduces to persistence, which is already a strong baseline at one hour. The softplus gaps guarantee non-crossing quantiles.

(e) Training objective. We minimise a pinball (quantile) loss, masked where the target is missing and weighted towards flood conditions:

$$\mathcal{L} = \frac{\sum_{b,i,q} w_{b,i} \, M_{b,i} \, \rho_q\big(y_{b,i} - \hat{y}_{b,i,q}\big)}{\sum_{b,i} M_{b,i}} + \lambda \|\theta\|^2, \quad \rho_q(u) = \max(qu, (q-1)u), \quad w_{b,i} = 1 + \kappa \, \mathbb{1}[y_{b,i} > \text{flood level}_i].$$

Training uses Adam, with early stopping on the validation period. The split is chronological (for example the first 70% of years for training, the next 15% for validation, the last 15% for testing), with a 13-hour gap at each boundary so windows do not overlap. After training, the 80% interval is widened by split-conformal calibration per station on the validation period, so its empirical coverage is 80%. We report MAE and pinball skill relative to persistence, interval coverage, and hit and false-alarm rates for threshold exceedance.

Why this design. (1) *Graph instead of a flat vector*: the river topology is known (F4). Hardwiring it removes impossible dependencies (S4 cannot drive S1) and lets the one temporal block and one spatial block be shared across nodes, which matters with only a few floods in the record (A1). (2) *T then S*: a node's own history is summarised first, so each spatial message carries a rich, time-aware description of the upstream state. (3) *Lag-attention instead of fixed lags*: travel times are unknown and state-dependent (F6, A9). A fixed-lag graph convolution would bake in a single lag, whereas attention restricted to physically plausible lags learns them and can shift them with the flow, at negligible cost because each node has at most two upstream neighbours. Attention over *all* space–time tokens is not needed: the graph is tiny and the plausible paths are known. (4) *Residual on persistence plus quantiles*: these make the model degrade gracefully and give the committee the uncertainty they need for flood decisions.

How each driver reaches each forecast.

Forecast	Upstream conditions	Tributaries	Rainfall	Reservoir releases
S1	own history (T block)	—	R1 via C (local runoff)	1 hop: past releases on the Res node, plus the announced future release f. The lag attention on Res→S1 picks the release that arrives in the next hour.
S2	S1 history at learned lag (1 hop)	T1 history (1 hop)	R1 via T1 and S1	2 hops via S1
S3	S2 at learned lag; S1 and T1 at 2 hops	latent U (1 hop). S3's own recent rise shows U's current inflow.	R2 via U and C; R1 via upstream nodes	3 hops
S4	S3 at learned lag; further upstream nodes at 2–4 hops	U and T1 through S3	R2 via U→S3→S4	4 hops; usually only relevant for long-lived release changes

Physical assumptions versus learned mappings.

Imposed by design (physical)	Learned from data
Flow direction and confluence locations (edges of A, A8)	How strongly each upstream node and rain gauge affects each station (attention values, MLPs)
Causality in time: no future level or rain inputs; the T block is causal	Travel times: lag weights α inside the plausible window, and their dependence on flow state
Travel time is positive, bounded, and grows with distance (L_{ji} cap and the $d_{ji}/\bar{v}$ prior, A9)	Rainfall–runoff response, including the U node's latent inflow, and the rain-to-node mapping C (initialised from A6)
Level continuity: the forecast is a change from the last level	Nonlinear stage response (rating-curve-like effects), seasonality, reliability of release schedules
The release schedule is an exogenous, known future input	Size and shape of the forecast uncertainty (quantile gaps)

We deliberately do *not* run a separate hydraulic simulator. Without cross-sections, rating curves or travel-time studies (requested in Section 6) it would need its own calibration and would add more unverified assumptions than it removes. The physics enters through structure and priors instead.

5. Operation and exceptional conditions

(i) Missing observations. Quality control (range, spike, flat-line and rate-of-change tests) marks bad values as missing. A missing reading is replaced by the last valid value, with mask = 0 and its age, so the model knows it is stale. During training we randomly mask about 5% of points and drop whole gauges for 1–6 h, which teaches the spatial block to rely on neighbours: for example, inferring S2 from S1 and T1 when S2 is offline. If a target station's own level is missing, y^{last} is its last valid value. The model still forecasts it from upstream information, and the conformal interval is calibrated separately for "stale" cases, so it widens automatically. If two or more main-river gauges are out for over 3 h, the system switches to a fallback (persistence plus a lagged regression on the available upstream gauge) and flags the forecast as degraded.

(ii) Delayed observations. Forecasts are never held back waiting for data. At t_0 + 2 min the model runs on whatever has arrived. Values not yet received are treated as missing with their true age, which is exactly how training was set up: delays are sampled from the empirical delay distribution, or from A3 if delay logs are not kept. Late data are included at the next issuance 10 minutes later. If late data change a forecast by more than a set tolerance (for example 5 cm), the current forecast is re-issued immediately with a "revised" flag.

(iii) Announced future releases. Schedules enter only through an *as-of* join. For every effective time e we use the most recent announcement with $a \le t_0$, so later revisions replace earlier ones and nothing announced after t_0 leaks in. The release in effect during $(t_0,\ t_0{+}60\text{ min}]$ forms X_{fut}, which reaches S1 (and onwards) through the Res node and the lag attention. If a scheduled change takes effect after t_0 but its travel time to S1 is longer than an hour, the attention simply puts no weight on it. The same as-of rule is used to rebuild historical training samples.

(iv) Scheduled versus actual releases. (1) The model is *trained* on schedules as known at issuance, not on what was actually released, so it learns how far schedules are typically honoured. (2) In the history window, the *recorded* release replaces the schedule as soon as it exists. The difference (recorded − scheduled) is an explicit input, so the model can discount a schedule that is not being followed. (3) If the recorded release deviates from the schedule by more than a tolerance, operators are alerted. The forecast is then run twice, once with the announced schedule and once with "current release persists", and both are published as a scenario band. Each run is a single forward pass, so this costs almost nothing. (4) S1's own level confirms the true release within its travel time, and all downstream forecasts correct automatically at the next issuance.

Feasibility of a 10-minute cycle. The network has roughly 10^5 parameters. One forecast costs about $4{\times}10^7$ multiply-adds in the T block (6 layers × 7 nodes × 72 steps × 3·32·64) and about $4{\times}10^6$ in the S block (4 rounds × 6 edges × 72 queries × ≤ 72 lags × 32). That is under 0.1 GFLOP, a few milliseconds on one CPU core. Even an ensemble of 5 models × 2 release scenarios takes well under a second. The cycle is dominated by data ingestion, quality control and alignment, which take seconds, leaving minutes of margin. Training is offline: about 5 years × 52,560 issuances per year is about $2.6{\times}10^5$ samples, which trains in minutes to hours on one GPU or CPU. We retrain quarterly and when drift monitoring triggers.

6. Uncertainty, limitations, and additional information

Uncertainty treatment. There are two sources. Aleatoric uncertainty is the irreducible variability given the inputs. It is represented by the predicted quantiles. Epistemic uncertainty is the model's own uncertainty, for example at floods larger than any in the record. It is estimated from the spread of an ensemble of 5 models trained with different seeds. The interval shown to users is the conformal-calibrated quantile interval, widened by the ensemble spread. Coverage is tracked separately for normal and flood periods.

Uncertainty / limitation	Impact on forecasts	Our response / what remains unresolved
Unmonitored tributary (between S2 and S3)	Its inflow is unobserved, so sudden rises from local storms appear at S3, and then S4, with no upstream warning. Errors are larger at S3/S4 than at S1/S2, especially in convective rain.	Latent node U driven by R2 rainfall. S3's history reveals the current contribution. Intervals at S3/S4 are calibrated separately and are wider. *Unresolved*: the first part of a sudden tributary flood cannot be anticipated better than R2 allows.
Changing travel times	A wrong lag shifts the predicted arrival of a rise. Peak-timing errors matter most at high flow.	State-dependent lag attention within physically capped windows, plus flood-weighted training. *Unresolved*: floods beyond the observed range may travel faster than anything seen, so lags there are extrapolated.
Future rainfall (unknown at issuance)	Rain in the next hour on small local catchments (around S1 and S4) can raise levels within the horizon.	Not modelled (A10). Wider intervals when recent rain intensity is high. Fixed by request 2 below.
Only two rain gauges	Convective cells can miss both gauges, so rain-driven rises are under-predicted.	The learned mapping C and the node histories compensate partly. Radar data would fix this (request 2).
Release deviations / unannounced spills	An unannounced change propagates to S1–S4 without warning.	Scenario band and alerts (Section 5 iv). S1 detects the change within its travel time.
Rare extremes (few floods in the record)	The quantiles may be poorly calibrated exactly when it matters most.	Flood weighting, ensemble spread, coverage monitored separately for floods, and a persistence-plus-routing fallback for comparison. Needs request 6.

Uncertainty / limitation	Impact on forecasts	Our response / what remains unresolved
Sensor faults, datum or rating changes, drift	Systematic bias at one station that also contaminates downstream inputs.	Quality control, masking augmentation, and drift monitoring of each station's residuals against its neighbours.
Non-stationarity (channel changes, new operating rules, snowmelt regimes)	Performance degrades slowly over time.	Season code, quarterly retraining, and rolling evaluation.
Backwater near confluences (A8 violated)	Small level errors at S2/S3 during high tributary flow.	A station's own history partly captures it. Reverse edges could be added if residuals show a pattern.

Additional information requests (in priority order; the system is designed to run without any of them):

#	Request	How it helps	Usable without it?
1	A temporary or permanent level/flow gauge on the unmonitored tributary, or a rain gauge in its catchment	Turns U into an observed node, the single biggest improvement for S3 and S4	Yes: U stays latent, with wider S3/S4 intervals
2	Weather-radar rainfall estimates and 0–2 h rainfall nowcasts	Captures rain the gauges miss and rain falling during the forecast hour	Yes: gauge rain only (A10)
3	Rating curves, channel cross-sections, any travel-time or tracer studies	Sharpens the travel-time prior and lag caps (A9). Would make a hydraulic-routing hybrid possible.	Yes: lags are learned within broad bounds
4	Telemetry metadata: data-arrival times, maintenance logs, quality flags, datum changes	Realistic delay and missingness simulation (A3, A4); removes known bad data	Yes: assumed delay and gap model
5	Reservoir operating rules, gate settings, and the history of schedule revisions and deviations	Better handling of schedule reliability and unannounced spills (A7)	Yes: reliability is learned from the record
6	Longest available historical record including past flood events, and the flood thresholds per station	More extreme events to train and calibrate on. Thresholds define the loss weights and evaluation (A1, A14).	Yes, but flood-time reliability is less certain
7	Air temperature and snow data for the headwaters; catchment boundaries (DEM)	Snowmelt contribution (A12); a better initial rain-to-node map C (A6)	Yes: season code and the learned C

Question 2 – Motion or Memory?

All code is in `Question2/` of the repository. Every number below is produced by the scripts listed in the README and read directly from `Question2/results/*.json`. Unless stated otherwise, results use the 1,000 test videos (indices 9,000–9,999) at 1× with $t = 1$. Learned models are reported as **mean (seed 0, seed 1)**, and figures use seed 0. **Hardware**: Apple M3 Pro (11-core CPU, 14-core GPU, 18 GB unified memory), PyTorch 2.10 on the MPS (Metal) backend, Python 3.10.

1. Setup

Data. `torchvision.datasets.MovingMNIST(root, split=None, download=True)` returns a uint8 tensor of shape (10000, 20, 1, 64, 64). The download was checked against torchvision's MD5. Videos 0–7,999 are used for training, 8,000–8,999 for validation and 9,000–9,999 for testing. Frames are kept as uint8 in memory and converted to float32 / 255 batch by batch, which gives exactly the same values as converting everything up front. Mirror is `flip(dims=(-1,))`, i.e. $(Mx)[i,j] = x[i,63-j]$. Rotation is `torch.rot90(x, k=a, dims=(-2,-1))`. A group element is stored as (m, a) and means $M^m R^a$ (rotate first, then mirror). Its inverse is itself when $m = 1$ and R^{-a} otherwise. The window function gathers frames $t + k \cdot \{0,\ldots,9\}$, keeps only targets with frame number ≤ 20, and returns a mask that both the loss and every metric use.

Table 1: Checks of the transformation and window functions, using the same functions the experiments call (`scripts/table1_checks.py`). All rows match.

Check	Expected	Yours
(a) max|M(M(x)) − x|	0	0 (torch.equal: True)
(b) max|R^4(x) − x|	0	0 (torch.equal: True)
(c) max|MRM(x) − R^{-1}(x)|	0	0 (torch.equal: True)
(d) bright pixel of M(p) (frame, row, column)	(7, 10, 43)	(7, 10, 43)
(e) bright pixel of R(p) (frame, row, column)	(7, 43, 10)	(7, 43, 10)
(f) 1× input / target frames	1–5 / 6–10	1–5 / 6–10
(g) 2× input / target frames	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19
(h) 3× input / target frames	1, 4, 7, 10, 13 / 16, 19	1, 4, 7, 10, 13 / 16, 19

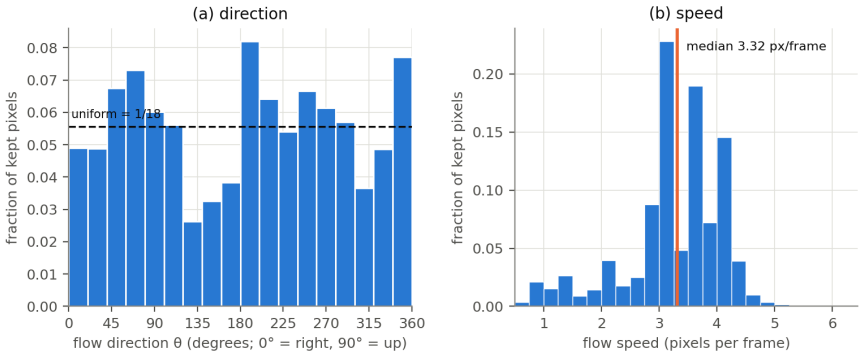


Figure 1: Dense Farneback optical flow on the first 50 training videos (950 consecutive frame pairs). Kept pixels have intensity > 0.2 in both frames and flow magnitude > 0.5 px. **103,050 pixels kept; median speed 3.32 px/frame.** (a) Flow direction $\vartheta = \text{atan2}(-d_y, d_x)$, with 0° = rightward and 90° = upward, as a fraction of kept pixels, in 18 bins of 20°. The dashed line is the uniform level 1/18. (b) Flow speed as a fraction of kept pixels. The orange line marks the median.

Q2.1

Directions. Figure 1(a) contains every direction. Each 20° bin holds between 2.6% and 8.2% of the kept pixels, against 5.6% for a uniform distribution, and no direction is missing or rare. Mirroring every training video would map each direction ϑ to $180° - \vartheta$. Rotating every video by 90° would map ϑ to $\vartheta + 90°$. To see how much this would change the histogram, we measured the total-variation (TV) distance between the histogram and its mirrored or rotated version: 0.13 for the mirror and 0.20 for the 90° rotation. The same distance between two *disjoint* sets of 50 training videos (0–49 versus 50–99) is already 0.09. So the transformed histograms differ from the original by roughly the amount expected from taking a different 50-video sample. On a supplementary set of 1,000 training videos the histogram becomes flatter (every bin between 4.0% and 6.9%), with TV distances of 0.08 (mirror), 0.10 (90° rotation) and 0.04 (180° rotation). The remaining unevenness is mild, consistent with directions being drawn at random, plus some direction-dependent bias in the flow estimator. Therefore **mirrored and rotated trajectories are already familiar as motion**: every direction they contain occurs often in training. What is *not* familiar after a mirror or rotation is the appearance of the digits (Q2.2).

Speeds. A typical digit moves about **3.3 px/frame** (median 3.32; interquartile range 3.0–3.9; 95th percentile 4.3; 99th percentile 4.7). Playing at 2× doubles the displacement between the frames the model sees, to about **6.6 px/frame** (≈ 8.6 at the 95th percentile). Playing at 3× gives about **10 px/frame** (≈ 13 at the 95th percentile). Almost no training pixel moves faster than 5 px/frame, so both speed-ups are *unfamiliar motion*.

Overlaps. Where digits overlap, one image patch contains two motions, so Farneback returns a blend or nonsense there. That adds noise to both histograms and pulls some speeds towards zero. It cannot change the answer, though. We recomputed the statistics after removing every frame pair in which the digits overlap at all (351 of 950 pairs, detected from the drop in the number of lit pixels). The median speed changes from 3.32 to 3.29 px/frame, the share of pixels slower than 2 px/frame from 9.0% to 8.7%, and the direction bins still span 2.7–8.3%.

Q2.2

Transformation	Motion statistics	Digit appearance
Mirror M	Essentially unchanged: directions are reflected ($\vartheta \to 180° - \vartheta$) onto directions that are equally common (Q2.1); speeds are unchanged.	**Changed**: mirrored digits (a reversed 2, 3, 7...) never occur in training.
Rotation R	Essentially unchanged: $\vartheta \to \vartheta + 90°$, again onto common directions; speeds are unchanged.	**Changed more**: rotated digits are even further from upright MNIST.
Speed-up k×	**Changed**: displacement per frame is multiplied by k, outside the training range.	Unchanged: the same upright digits.

This makes the three transformations a controlled experiment. Each one changes mostly one factor and leaves the other fixed, and the motion rule itself (constant velocity with elastic bounces in a square box) is valid under all three. A model that has truly learned motion should cope with all of them. If it fails only under M and R, the cause is appearance (its motion rule is tied to what upright digits look like). If it fails only under speed-up, the cause is the motion itself, which is either unfamiliar or beyond the network's reach. Comparing M with R also varies the *size* of the appearance change while the motion statistics stay the same.

2. Baseline: ConvLSTM

Implementation (`mm/models.py`). Each step does the following: `pixel_unshuffle(x, 4)` (1×64×64 → 16×16×16); ConvLSTM layer 1 (input 16 channels, 64 hidden, 3×3, padding 1); ConvLSTM layer 2 (input $h^{(1)}_\tau$, 64 hidden); a 1×1 convolution 64 → 16; `pixel_shuffle(·, 4)`; and a sigmoid. Each layer uses one `Conv2d(in+64, 4·64, 3, padding=1, bias=True)` on $[u_\tau,\ h_{\tau-1}]$. Its output is split into $[i, f, o, g]$ and plugged into Eqs. (3)–(4), with no peepholes, no normalisation and default initialisation. States start at zero. The network rolls out for 9 steps: steps 1–5 read the input frames, steps 6–9 read the network's own previous prediction (no teacher forcing, gradients flow through the fed-back frames), and the outputs of steps 5–9 form $\hat{Y}$. The trainable parameter count is 184,576 + 295,168 + 1,040 = **480,784**, matching the handout.

Training (`mm/train.py`). Adam with learning rate 10^{-3}, batch size 32, 30 epochs and no schedule; the final-epoch weights are kept. Each epoch is one shuffled pass over the 8,000 training videos with one random window per video. The loss is the squared error averaged over the pixels and the *available* target frames of each video, then over the batch. `torch.manual_seed(seed)` controls initialisation, and a separate `torch.Generator` seeded with the same value controls shuffling, window, speed and symmetry sampling. Training time per epoch counts only the optimisation loop (with device synchronisation). The validation monitoring every 5 epochs is excluded.

Table 2: Per-horizon test MSE, $E^{(5)}$ and trainable parameters at 1×, $t = 1$. Model A is mean (seed 0, seed 1).

Predictor	MSE_1	MSE_2	MSE_3	MSE_4	MSE_5	
Model A	**0.0172** (0.0175, 0.0169)	**0.0206** (0.0209, 0.0203)	**0.0228** (0.0231, 0.0225)	**0.0245** (0.0249, 0.0242)	**0.0261** (0.0265, 0.0258)	**0.0223** (0
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	

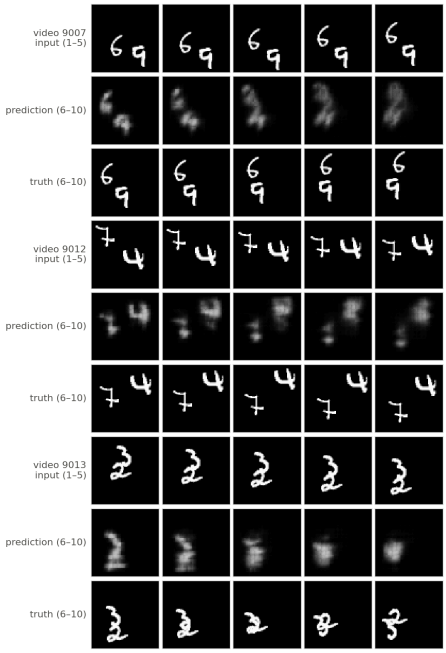


Figure 2: Model A (seed 0) on three test videos at 1×. For each video: input frames 1–5, the prediction for frames 6–10, and the ground truth for frames 6–10. Videos 9007 and 9012 are chosen automatically as the first test videos whose digits are clearly separated in the input. In video 9007 the digits approach each other. Video 9013 is the first test video whose digits overlap strongly in the target frames.

Q2.3 — Reach of model A.

Following the stated conventions, track how far (centre to centre, in original pixels) information can move between entering the network at step τ and leaving it as $\hat{x}_{\tau+1}$:

1. **Patch folding**: `pixel_unshuffle(4)` turns each 4×4 patch into one cell of a 16×16 grid. It moves nothing, but from here on **one cell = 4 px**.
2. **ConvLSTM layer 1**: its single 3×3 convolution (stride 1, padding 1) on $[u_\tau, {h^{(1)}}_{\tau-1}]$ lets each output cell read its 8 neighbours, so information moves at most **1 cell = 4 px**. The gate, cell and hidden updates in Eq. (4) are element-wise and move nothing.
3. **ConvLSTM layer 2**: its 3×3 convolution on $[{h^{(1)}}_\tau, {h^{(2)}}_{\tau-1}]$ moves at most another **1 cell = 4 px**.
4. **Output**: the 1×1 convolution, `pixel_shuffle` and the sigmoid move nothing.

Within a single step the possible paths are:

- new input $u_\tau \to \text{conv}_1 \to \text{conv}_2$: 2 cells = 8 px;
- previous layer-1 state ${h^{(1)}}_{\tau-1} \to \text{conv}_1 \to \text{conv}_2$: 8 px;
- ${h^{(2)}}_{\tau-1} \to \text{conv}_2$: 4 px;
- cell states $c^{(1)}$, $c^{(2)}$, which enter element-wise: 4 px and 0 px.

The longest is therefore **reach = 2 × 1 cell × 4 px = 8 px per time step** along a row or column. Along a diagonal a 3×3 kernel also reaches the corner cell, so the Euclidean reach is $8\sqrt{2} \approx 11.3$ px. Recurrence does not add to the one-step reach. It only lets information travel further over *several* steps, at most 8 px more per step. So a digit that jumps much more than about 8 px between frames lands outside the neighbourhood from which the remembered position (held in $h^{(1)}$, $h^{(2)}$) can be moved in one update. The network then cannot carry the digit's state along with it.

Q2.4

Against the reference predictors (Table 2): model A's $\text{E}^{(5)} = 0.0223$ is about half of Blank's (0.0426) and about a third of Copy-last's (0.0641), and it beats both at every horizon. Copy-last is *worse* than Blank because a moving digit copied in the wrong place is penalised twice: once where it is drawn but is not, and once where it should be but is not. Model A therefore really predicts where the digits go, which neither reference can do.

Error over horizons: MSE grows monotonically from 0.0172 at $h = 1$ to 0.0261 at $h = 5$, about +52%. The growth slows with h (+0.0034, +0.0022, +0.0017, +0.0016 per step). Figure 2 shows why. Uncertainty about position and shape builds up, and under a squared-error loss the best response to uncertainty is to *average* over possibilities, so predictions become smoother and dimmer with each step. They are also fed back as inputs, so the network then works from its own blurred frames. Even at $h = 5$ model A is still 39% below Blank, so the predicted blobs remain in roughly the right places.

Overlaps: in video 9013 the 3 and the 2 overlap in the input and separate in the target. Model A does not separate them. The two digits melt into one bright blob, which barely moves and loses all digit shape by $h = 4$–5. In video 9007 the 6 and the 9 approach each other. The prediction pulls them together into a single vertical smear, whereas in the truth both digits move up and stay distinct. Once the two digits share pixels, the model can no longer tell which stroke belongs to which digit or which way each is moving. It falls back to a blurry average of both, which is what the squared-error loss rewards under that ambiguity.

Q2.5 — predictions, reproduced verbatim as first written

Committed to the repository before model A was trained or any Testing-the-Baseline experiment was run: `Question2/predictions/Q2.5_predictions.md`, commit `48c2694`.

Mirror test. Model A will mostly pass on *motion*: mirrored trajectories use directions that are common in training, and the convolutional motion rule is local. Mirrored digits, however, are new shapes. I expect E_M to be somewhat higher than the original E(5) (roughly 10–30% higher), with the extra error in the later horizons as shape distortion or blur rather than wrong motion. C_M will be clearly non-zero but well below E(5).

Rotation test. The same reasoning holds for the motion, but rotated digits look even less familiar than mirrored ones. I expect the rotation test to be "passed less well" than the mirror test: E_R > E_M and C_R > C_M. The difference should come from appearance, not from motion.

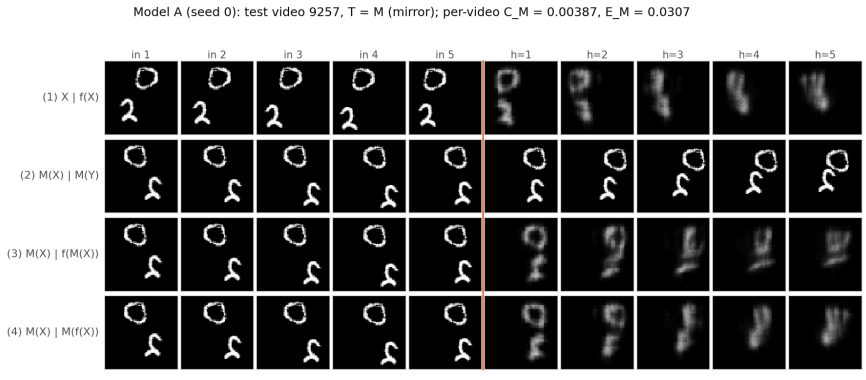
Speed. With an 8 px/step reach and 3.3 px/frame typical motion: 1×: comfortably inside reach. 2× (≈ 6.6 px/frame): still mostly inside the reach but near its edge, and faster than anything seen in training. I expect noticeable degradation: E(2) clearly higher than at 1×, but predictions still recognisably move the digits. 3× (≈ 10 px/frame, often above 8 px): beyond the one-step reach for most videos. **I predict model A breaks down at 3×**: digits will lag behind or stall near their last input position and blur or fade out. E(2) at 3× will approach or exceed Copy-last.

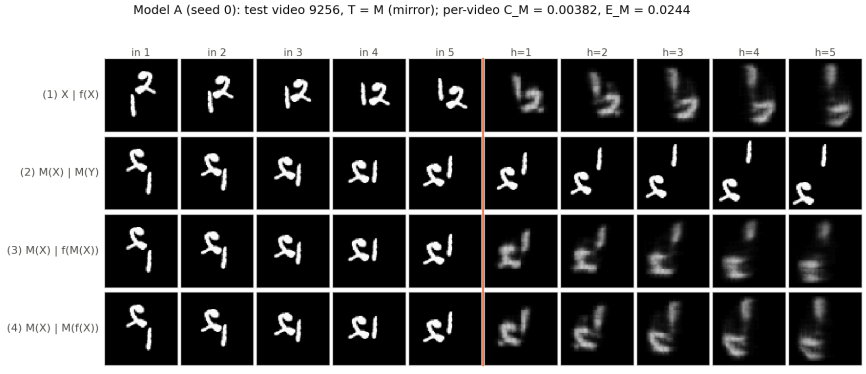
3. Testing the Baseline

Mirror and rotation

Table 3: Accuracy and consistency under mirror (M) and rotation (R) at 1×, $t = 1$, on the 1,000 test videos. Model A is mean (seed 0, seed 1).

Predictor	$E^{(5)}$	C_M	E_M	C_R	E_R
Model A	0.0223 (0.0226, 0.0220)	0.00166 (0.00160, 0.00172)	0.0229 (0.0231, 0.0226)	0.00283 (0.00250, 0.00317)	0.0239 (0.0241, 0.0238)
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641





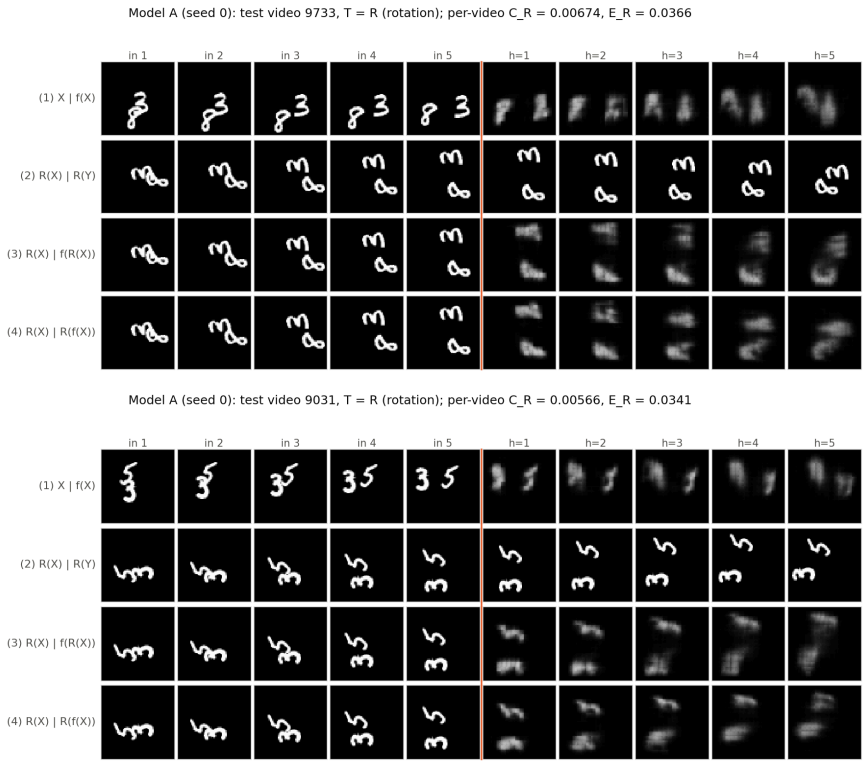


Figure 3: The two test videos with the largest per-video C_M (a: 9257, b: 9256) and the two with the largest C_R (c: 9733, d: 9031), for model A (seed 0). No video is in both top-two lists. Rows: (1) $X \mid f(X)$; (2) $T(X) \mid T(Y)$; (3) $T(X) \mid f(T(X))$; (4) $T(X) \mid T(f(X))$. Columns 1–5 are the input frames and columns 6–10 the forecast horizons $h = 1$–5. The orange line separates input from forecast. Animated GIFs of the same four rows, stepping through the five inputs and then the five horizons: `Question2/gifs/fig3_modelA_M_v9257.gif`, `Question2/gifs/fig3_modelA_M_v9256.gif`, `Question2/gifs/fig3_modelA_R_v9733.gif`, `Question2/gifs/fig3_modelA_R_v9031.gif`.

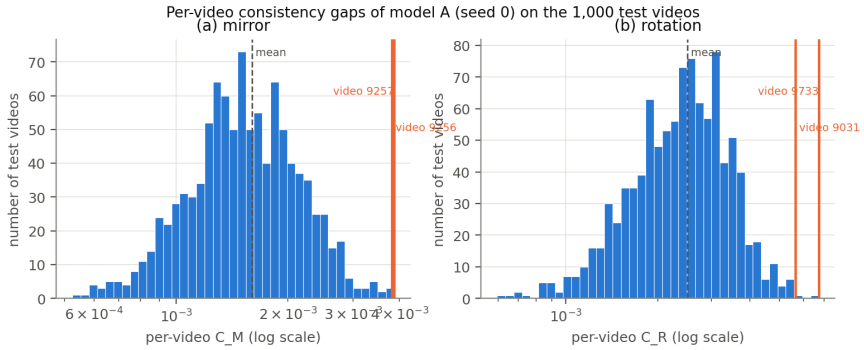


Figure 4: Per-video consistency gaps of model A (seed 0) over the 1,000 test videos, on a log x-axis: (a) C_M, (b) C_R. The dashed line is the mean. The orange lines mark the videos shown in Figure 3, which are the two largest values of each gap.

Q2.6

Blank and Copy-last have $C_M = C_R = 0$ exactly. An all-zero frame is unchanged by any pixel permutation, and "repeat the last frame" commutes with any transformation applied to all frames. Yet they are the *worst* predictors in Table 3 ($E_T = 0.0426$ and 0.0641, against 0.0229–0.0240 for model A). So a small consistency gap on its own says nothing about whether a model has learned motion. Any predictor that ignores the content in a symmetric way, or does nothing at all, is perfectly consistent. C_T only asks whether the model treats X and $T(X)$ in the same way, not whether that way is right. The transformed error $E_T = \mathrm{MSE}(f(T\ X),\ T\ Y)$ supplies the missing half: is the prediction on the transformed clip actually correct? The useful reading needs both together. A low C_T together with E_T close to the model's own $E^{(5)}$ means the same accurate rule is applied to transformed scenes. That is the case for model A here (E_M is only 2.6% and E_R 7.5% above its $E^{(5)}$). A low C_T with a high E_T just means "consistently wrong".

Q2.7 — Video 9733 under rotation (Figure 3, largest C_R)

In the original clip an 8 moves left towards the left wall and a 3 moves right. After the 90° anticlockwise rotation, the left wall becomes the *bottom* wall. The rotated 8 (an "∞") travels down-left, hits the bottom wall at about $h = 2$, and bounces back up. The rotated 3 (an "m") moves straight down at about 3.5 px/frame. Below, (row, column) is the intensity centroid of each digit in the 64×64 frame, from row (2) $R(Y)$, row (3) $f(R\ X)$ and row (4) $R(f(X))$:

	h=1	h=2	h=3	h=4	h=5
"∞" (rotated 8), truth	(46.6, 42.3)	(48.6, 39.3)	(46.6, 37.3)	(43.6, 34.3)	(40.6, 33.8)
"∞", row 3: $f(R\ X)$	(46.5, 40.6)	(48.1, 38.6)	(48.9, 36.2)	(49.0, 33.7)	(47.6, 32.1)
"∞", row 4: $R(f(X))$	(46.6, 40.7)	(47.9, 38.4)	(47.6, 36.3)	(45.9, 34.4)	(42.8, 33.3)
"m" (rotated 3), truth	(12.7, 40.0)	(16.7, 41.0)	(20.7, 42.0)	(23.7, 42.0)	(26.9, 42.7)
"m", row 3	(12.1, 38.6)	(11.7, 39.4)	(15.8, 40.4)	(20.1, 40.2)	(22.2, 40.8)
"m", row 4	(13.2, 38.5)	(15.6, 39.1)	(19.7, 39.6)	(22.8, 39.6)	(25.0, 39.5)

Rows 3 and 4 disagree in two specific ways:

1. **The bounce comes late and is almost missed.** On the rotated clip (row 3), model A keeps pushing the "∞" into the bottom wall: its row stays at 48–49 from $h = 2$ to 4, and by $h = 5$ it has risen only 1.4 px. The truth rises 8 px after bouncing at $h = 2$. On the original clip the same model handles the same bounce off the left wall correctly, so row 4 bounces at $h = 2$–3 and rises 5 px. Its horizontal motion (the column) is right in both rows. Only the reflection is lost.
2. **The other digit stalls and fades.** In row 3 the "m" does not move at all for the first forecast step (row 12.1 → 11.7, against a true +4 px). It then moves down more slowly and ends 4.7 px behind the truth. Its peak brightness drops from 0.73 to 0.48, so it is turning into a dim blur. In row 4 the 3 moves steadily at about 3 px/frame, 2 px behind the truth.

So for this clip the model's account of the motion changes from "an 8 bounces off a wall and a 3 moves away" to "a sideways 8 slides along the floor and a sideways 3 hesitates and dissolves". The motion itself (a bounce off a wall, constant downward velocity) is just as common in training as the original. What is new is only that the shapes doing the moving are lying on their side.

Q2.8

Yes, rotation is harder than mirroring for model A, and the effect is consistent across both seeds. $E_R - E^{(5)} = 0.0017$ (+7.5%) against $E_M - E^{(5)} = 0.0006$ (+2.6%). C_R (0.0028) is about 1.7× C_M (0.0017), and the C_R histogram in Figure 4 has a heavier right tail.

The cause is appearance, not motion. By Q2.1, neither transformation creates unfamiliar *motion*: every direction a mirrored or rotated trajectory contains is common in training, and speeds are unchanged. On 1,000 videos, the 90° rotation moves the direction histogram only slightly more than the mirror does (TV 0.10 against 0.08). Both are small next to how well every direction is represented. By Q2.2, what the two transformations change is the digits' *appearance*, and rotation changes it much more. A mirrored digit is still upright: it keeps its height-to-width ratio and its mostly vertical strokes. Only its left–right orientation flips, and some digits (0, 1, 8) are nearly mirror-symmetric anyway. A rotated digit lies on its side. It is wider than it is tall, its long strokes become horizontal, and no rotated digit resembles any upright training digit. Model A's motion "rule" is implemented by convolution filters tuned to upright strokes, and the extra error and inconsistency grow with how far the appearance moves from the training digits. That points to a motion rule that is partly entangled with appearance. The cleaner test is model B (Section 4): if symmetry augmentation removes the gap between E_M and E_R without changing the motion statistics, appearance is confirmed.

Faster motion

Table 4: Model A and Copy-last at 1×, 2× and 3× ($t = 1$). $E^{(2)}$ uses only the first two horizons, so it is comparable across speeds. At 3× only two target frames exist (16 and 19), so $E^{(5)}$ is not defined there. For reference, Blank scores $E^{(2)} = 0.0426$ at every speed.

Predictor	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	$E^{(5)}$ 1×	$E^{(5)}$ 2×
Model A	0.0189 (0.0192, 0.0186)	0.0443 (0.0428, 0.0458)	0.0515 (0.0502, 0.0528)	0.0223 (0.0226, 0.0220)	0.0488 (0.0465, 0.0510)
Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

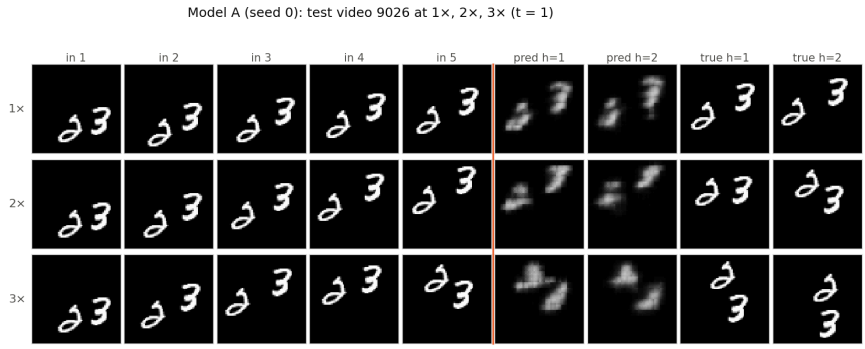


Figure 5: Model A (seed 0) on test video 9026 (chosen automatically as the first test video whose two digits form two separate blobs in every displayed frame at all three speeds) at 1×, 2× and 3× ($t = 1$): the five input frames, the first two predicted frames and the first two true future frames. The digits do not overlap in any of the displayed true frames.

Q2.9

Model A falls apart already at 2×, not at 3×. $E^{(2)}$ rises from 0.0189 at 1× to 0.0443 at 2× (2.3×) and 0.0515 at 3×. At 2× it is no better than predicting an empty frame (Blank: 0.0426), and at 3× it is worse than Blank. Both seeds agree (0.0428 and 0.0458 at 2×). It still beats Copy-last at every speed, but only because Copy-last is a very weak bar.

Agreement with the reach estimate: no. The typical 2× displacement (6.6 px/frame) is *inside* the 8 px reach, so Q2.3 and my Q2.5 prediction expected 2× to degrade but still work. The estimate missed two things. (1) **Reach is only an upper bound on what the architecture *could* do.** Whether the trained weights actually move content that far depends on training. Model A only ever saw displacements up to about 5 px/frame, so its filters implement those, and nothing in training asked them to move a digit 6–10 px per step. (2) **One-way reach is not the limit on relating two positions.** To track a digit, the old position (held in the recurrent state) and the new evidence (the input frame) must *meet* in some cell. They can travel towards each other from both ends: the input 2 cells to the output, and the layer-2 state 1 cell. So positions up to about 3 cells = 12 px apart can be compared in a single step, which already covers 3× (≈ 10 px). Section 4 confirms both points: speed augmentation alone (model C, same 8 px reach) repairs most of the 2× *and* 3× failure.

What the failure looks like (Figure 5, video 9026; intensity centroids of each digit):

- **Stalling.** At 2× the true digits move about 7 px per step. Model A moves them only about 2 px (the 3: true row 14.1 → 21.1 at $h = 1$, predicted 14.1 → 12.1; the 2: true column 16.2 → 20.2, predicted 16.2 → 15.6). At 3× the 3 should move 10 px down in one step and 21 px in two, but the prediction moves it 3 px and then 4.5 px. Stalling suggests the model is applying a *memorised typical step* from the 1× training data (a prior on how far digits move) instead of measuring the velocity in the input frames.
- **Missed bounces.** At 2× the 3 hits the top wall and bounces down, but the prediction keeps nudging it upwards into the wall. A model that has not tracked the velocity cannot know when the wall will be reached.
- **Blur.** The predicted blobs cover 1.5–2× the true digit area (for example 267 against 137 px at 3×), and their peak intensity drops to 0.7–0.8. That is the squared-error-optimal hedge when the model is unsure where the digit is.
- **A faint duplicate.** At 2×, $h = 2$, a ghost of the 2 appears about 10 px away from the stalled copy. The model spreads probability mass between where its prior says the digit should be and where the input evidence points. No digit disappears entirely in this example, but at 3× the shapes dissolve into blobs that are no longer recognisable.

Q2.10

No, Table 4 alone cannot separate the two explanations. Model A was trained only at 1×, where almost all motion is below 5 px/frame (Figure 1b), and its reach is 8 px. At 2× and 3× the motion is therefore *both* unfamiliar and close to or beyond the reach. Both hypotheses predict exactly the observed result, a collapse at 2× and 3×. The only hint is that model A already fails at 2×, where the typical displacement (6.6 px) is *inside* its reach. That suggests unfamiliarity is at least part of the story, but it cannot rule reach in or out at 3×.

Experiment that separates them: a 2×2 design that changes one factor at a time.

1. **Fix unfamiliarity, keep the reach**: retrain the *same* architecture with speed augmentation (model C). If C becomes good at 2× *and* 3×, the failure was purely unfamiliar motion. If C recovers at 2× (within reach) but stays poor at 3× (beyond reach), the remaining 3× failure is architectural.
2. **Then increase the reach, keep the data**: train a larger-reach network on exactly the same speed-augmented data (model D). If D fixes 3× where C could not, reach is confirmed as the cause.

A cheaper diagnostic on the existing models is to bin the test videos by their measured per-frame displacement at 2×/3× and plot error against displacement. An architectural limit predicts a sharp rise in error near the reach (about 8 px), however much data is used. Unfamiliarity predicts errors that rise from the edge of the training range (about 5 px) and fall back once such speeds are added to training.

4. What Can Data Fix?

Models B, C and D are trained under exactly the same protocol as model A (`mm/train.py`), with these changes. **B**: for each training window, g is drawn uniformly from the 8 elements of G (including the identity) and the same g is applied to the input and the target. **C**: for each window, k is drawn uniformly from {1, 2, 3}, then t uniformly from that speed's allowed starts ({1..14}, {1..8}, {1, 2}). Only the existing target frames enter the loss, and each video is weighted equally. **D**: the same data as C with a larger-reach architecture (Q2.12). **D+B** (optional): D's architecture trained with *both* speed and symmetry augmentation. It is reported as an extra row of Table 5 and is discussed in Section 5.

Q2.11 — predictions, reproduced verbatim as first written

Committed after Testing the Baseline and before any of B, C, D (or a D candidate) was trained: `Question2/predictions/Q2.11_predictions.md`, commit `b5b060a`.

Model B (symmetry augmentation) will shrink C_M and C_R and bring E_M and E_R down to its own E(5), fixing the appearance-tied part of the mirror/rotation failure, but it will not help at 2× or 3×. Model C (speed augmentation, same architecture) will clearly improve 2× because about 6.6 px/frame is still within the 8 px reach, but it will stay poor at 3× (about 10 px/frame) because that motion exceeds the network's one-step reach. Model D (larger reach, speed augmentation) will be the only model that also works at 3×, with E(2) at 3× much closer to its 1× value, while none of B–D should lose more than a little accuracy on the original 1× clips.

Table 5: Models A–D and the optional D+B on the 1,000 test videos. $E^{(5)}$, C_T and E_T are at 1× with $t = 1$. $E^{(2)}$ is at 1×, 2× and 3×. Entries are mean (seed 0, seed 1). Training time is the mean wall-clock time per epoch over the 30 epochs. Inference time is the wall-clock time to predict all 1,000 original test videos at batch size 32 (best of 3 repeats). Hardware: Apple M3 Pro GPU (MPS).

(a) Accuracy and consistency at 1×

Model	$E^{(5)}$ 1×	C_M	E_M	C_R	E_R
A	0.0223 (0.0226, 0.0220)	0.00166 (0.00160, 0.00172)	0.0229 (0.0231, 0.0226)	0.00283 (0.00250, 0.00317)	0.0239 (0.0241, 0.0238)
B	0.0230 (0.0237, 0.0223)	0.00122 (0.00123, 0.00120)	0.0230 (0.0237, 0.0223)	0.00140 (0.00148, 0.00132)	0.0229 (0.0236, 0.0222)
C	0.0267 (0.0271, 0.0264)	0.00131 (0.00152, 0.00111)	0.0269 (0.0273, 0.0266)	0.00257 (0.00280, 0.00235)	0.0278 (0.0281, 0.0275)
D	0.0274 (0.0269, 0.0278)	7.17×10^{-4} (8.49×10^{-4}, 5.86×10^{-4})	0.0275 (0.0271, 0.0280)	0.00189 (0.00172, 0.00207)	0.0284 (0.0279, 0.0289)
D+B	0.0277 (0.0276, 0.0279)	5.26×10^{-4} (7.01×10^{-4}, 3.51×10^{-4})	0.0278 (0.0276, 0.0279)	4.72×10^{-4} (6.06×10^{-4}, 3.37×10^{-4})	0.0278 (0.0276, 0.0280)

(b) Speed generalisation and cost

Model	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Params	Train s/epoch	Inference (1,000 videos)
A	0.0189 (0.0192, 0.0186)	0.0443 (0.0428, 0.0458)	0.0515 (0.0502, 0.0528)	480,784	23.7 s (23.8 s, 23.6 s)	0.873 s (0.873 s, 0.874 s)
B	0.0194 (0.0201, 0.0188)	0.0436 (0.0437, 0.0435)	0.0500 (0.0506, 0.0495)	480,784	30.1 s (30.2 s, 29.9 s)	0.935 s (0.928 s, 0.942 s)
C	0.0233 (0.0237, 0.0229)	0.0288 (0.0288, 0.0287)	0.0333 (0.0332, 0.0334)	480,784	23.6 s (23.6 s, 23.6 s)	0.932 s (0.933 s, 0.931 s)
D	0.0245 (0.0240, 0.0250)	0.0290 (0.0285, 0.0296)	0.0329 (0.0326, 0.0333)	480,784	25.8 s (25.7 s, 25.8 s)	1.05 s (1.04 s, 1.05 s)
D+B	0.0248 (0.0247, 0.0250)	0.0294 (0.0295, 0.0293)	0.0329 (0.0328, 0.0329)	480,784	31.6 s (32.2 s, 31.0 s)	0.988 s (0.989 s, 0.986 s)

Q2.12 — Model D and its reach

How D was chosen. Three convolutional-recurrent designs with a strictly larger one-step reach and at most 721,176 parameters were fixed in advance (`mm/models.py: D_CANDIDATES`). The selection rule was fixed with them: the lowest mean *validation* $E^{(2)}$ over 1×, 2× and 3× at the final epoch (`scripts/select_D.py`). Both were committed before any candidate was trained. Each candidate was then trained with the full model-D protocol (speed augmentation, seed 0, 30 epochs). The winner's seed-0 run is model D seed 0, and seed 1 was trained afterwards with the same design. The test split was not used.

Table D: Model-D candidates on the **validation** split (seed 0, final epoch). One-step reach uses the Q2.3 convention.

Candidate	Change vs. A	Reach	Params	val $E^{(5)}$ 1×	val $E^{(2)}$ 1×	val $E^{(2)}$ 2×	val $E^{(2)}$ 3×	mean val $E^{(2)}$
dil12 ✓ chosen	layer 2 dilated ×2 (3×3)	1 + 2 cells = 12 px	480,784	0.0266	0.0238	0.0283	0.0320	0.0281
k55h46	5×5 kernels, 46 hidden	2 + 2 cells = 16 px	709,520	0.0395	0.0395	0.0395	0.0395	0.0395
p8	8×8 patches (8×8 grid)	2 cells × 8 px = 16 px	594,496	0.0291	0.0275	0.0292	0.0312	0.0293

The 5×5-kernel design collapsed in epoch 2 to the trivial "almost-dark frame" solution (training loss stuck at 0.0402 for all 30 epochs, and the same validation error at every speed). The output sigmoid saturated and gradients vanished. Because the protocol forbids tuning the learning rate, the design was rejected rather than rescued. The 8×8-patch design has the flattest error across speeds but loses detail at 1× (its grid cells are 8 px wide). The dilated design wins on the pre-registered criterion.

Model D (chosen design "dil12"). Everything is identical to model A (pixel_unshuffle(4), two ConvLSTM layers with 64 hidden channels and Eqs. 3–4, a 1×1 head, pixel_shuffle, sigmoid, the 9-step rollout without teacher forcing, and the same training protocol), except that **the 3×3 gate convolution of layer 2 is dilated by 2** (dilation 2, zero padding 2, so the grid stays 16×16). Dilation spreads the same nine taps over a 5×5 window at offsets {−2, 0, +2} cells, so the parameter count is unchanged at **480,784** (= model A, well under the 721,176 limit). This keeps model D a clean single-factor change from model C: only the reach differs.

Reach (same calculation as Q2.3; one cell = 4 px; only the two 3×3 convolutions move information):

- Layer 1, a 3×3 kernel with dilation 1, moves information at most **1 cell = 4 px**.
- Layer 2, a 3×3 kernel with dilation 2, moves information at most **2 cells = 8 px**.
- Path u_τ (or ${h^{(1)}}_{\tau-1}$) → conv_1 → conv_2 → $\hat{x}_{\tau+1}$: 1 + 2 = **3 cells = 12 px**. Path ${h^{(2)}}_{\tau-1}$ → conv_2: 2 cells = 8 px. Cell states enter element-wise.

So **reach(D) = 12 px per step > reach(A) = 8 px** along a row or column (17 px against 11.3 px along the diagonal). The offsets that the two layers combine, {−1, 0, 1} + {−2, 0, 2} = {−3, ..., 3} cells, are contiguous. Every displacement up to 12 px can therefore be carried in one step, with none of the gaps that stacking two dilated layers would leave. That covers the median 3× displacement of about 10 px (Q2.1). The 8×8-patch candidate would have reached 16 px, but it lost on the validation criterion.

Q2.13 — Which failure is which?

Failure of model A	Label	Supporting comparison (Table 5, mean of two seeds)
Mirror/rotation inconsistency, larger for rotation (C_M = 0.0017, C_R = 0.0028; E_R 7.5% above $E^{(5)}$)	**Appearance**	**A → B** changes only what the digits look like in training (motion statistics are unchanged, Q2.1). C_R halves (0.0028 → 0.0014), C_M drops (0.0017 → 0.0012), and the rotation penalty disappears: E_M = 0.0230 and E_R = 0.0229 against $E^{(5)}$ = 0.0230. The speed-augmented models C and D, which saw no new appearances, keep C_R at 0.0019–0.0026. Adding symmetry augmentation to D (**D → D+B**) cuts C_R from 0.0019 to 0.00047 while leaving every speed error unchanged, which confirms the same label.
Collapse at 2× ($E^{(2)}$ 0.0443, about Blank)	**Unfamiliar motion**	**A → C** keeps the architecture and its 8 px reach and only adds faster motion to training. $E^{(2)}$ at 2× falls 35% (0.0443 → 0.0288). B, which saw no faster motion, stays at 0.0436.
Collapse at 3× ($E^{(2)}$ 0.0515, worse than Blank)	**Mainly unfamiliar motion**; architectural reach is not the binding limit here	**A → C**: 0.0515 → 0.0333 (−35%) with the *same* reach. **C → D** (reach 8 → 12 px, same data): 0.0333 → 0.0329, inside the seed-to-seed spread (C: 0.0332 and 0.0334; D: 0.0326 and 0.0333).

If C had stayed bad at 3× while D did well, that would have isolated an architectural limit. C and D see identical data, so data cannot explain a difference between them, and the only remaining difference would be how far information can travel in one step. **Our runs show something different**: C already fixes 3× almost as well as D. The reason is the point made in Q2.9. Relating a digit's remembered position to its new position needs the two pieces of information to *meet*, and in model A they can meet across about 12 px (2 cells from the input plus 1 cell from the layer-2 state). That already covers the ≈ 10 px displacements at 3×. So 3× was unfamiliar, not out of reach. A gap remains in both models: $E^{(2)}$ at 3× is 43% (C) and 34% (D) above their own 1× value. D's ratio is slightly better, which is consistent with reach mattering a little at the 95th-percentile 3× displacements (≈ 13 px). Part of the gap is also intrinsic: one 3× step spans three real frames, with more bounces and more uncertainty, and Copy-last also worsens from 0.0548 to 0.0703.

Why more data cannot carry information further than the architecture allows. The reach is a property of the computation graph, not of the weights. The prediction at a pixel is a function only of the input and state values inside its one-step neighbourhood. For anything outside

it there is no path, so the derivative of the output with respect to those inputs is exactly zero for *every* weight setting. Training data can only change weights. It can teach the network to use the connections it has, as C does, but it cannot create connections that do not exist. No amount of data lets a 3×3 stack see 20 px away in one step. Only a change to the architecture (larger or dilated kernels, bigger patches, downsampling) can do that.

Q2.14 — Did the fixes cost accuracy on the original 1× clips?

B: essentially no. $E^{(5)}$ at 1× is 0.0230 against 0.0223 for A (+3%), which is inside the seed spread (A: 0.0220–0.0226; B: 0.0223–0.0237). In return it buys consistency and an equal error on mirrored and rotated clips. That is clearly worth it. **C and D: yes.** $E^{(5)}$ at 1× rises to 0.0267 (+20%) and 0.0274 (+23%), and $E^{(2)}$ at 1× from 0.0189 to 0.0233 and 0.0245. With the epoch budget fixed, only about one window in three is now a 1× window, and the same network must cover a three-times-wider range of motions, so predictions get blurrier at 1×. Averaged over the three speeds, however, $E^{(2)}$ drops from 0.0382 (A) to 0.0284 (C) and 0.0288 (D), about 25% lower, and the catastrophic failures at 2× and 3× are gone. **It is worth it whenever speeds other than 1× can occur**, which is the point of a model that has "learned motion". If the deployment only ever sees 1× clips, the speed augmentation is a pure loss. D's extra reach did not pay for itself in our runs: it is slightly worse than C at 1×, equal at 3×, and about 9% slower per epoch.

5. The Final Model

Symmetrised forecaster (`mm/models.py: Symmetrised`). The eight transformed copies $g(X)$, $g \in G$, are stacked into one batch of $8B$ videos. The trained model runs once on that batch, each output is mapped back with g^{-1}, and the eight results are averaged (Eq. 7). Every transformation acts frame by frame on all five frames, so the 9-step rollout and the feedback of predictions happen *inside* each transformed copy.

Why $\bar{f}$ has $C_M = C_R = 0$ exactly, for any f.

Let $T \in G$ (in particular $T = M$ or R). Then

$$\bar{f}(TX) = \frac{1}{8}\sum_{g\in G} g^{-1}\big(f(gTX)\big) \overset{h=gT}{=} \frac{1}{8}\sum_{h\in G}(hT^{-1})^{-1}\big(f(hX)\big)$$

$$= \frac{1}{8}\sum_{h\in G} T\,h^{-1}\big(f(hX)\big) = T\Big(\frac{1}{8}\sum_{h\in G} h^{-1}\big(f(hX)\big)\Big) = T\big(\bar{f}(X)\big).$$

The substitution $h = gT$ is valid because G is a group: right-multiplying by T just permutes its eight elements, so the sum runs over the same set. $(hT^{-1})^{-1} = T\,h^{-1}$, and T can be taken out of the average because a pixel permutation is linear. Hence $\bar{f}(TX) = T(\bar{f}(X))$ for every input, so $C_T = 0$. Nothing about f was used, so this holds for any forecaster, trained or not. In floating point the two sides sum the same eight terms in a different order, and they go through the GPU in different batch positions, so the measured values are at rounding-error level (Table 6), not exactly zero.

Choice of final model: symmetrised D (the handout's default), confirmed against the optional symmetrised D+B, which we also trained. Choosing between the two on the test set would break the rule against using the test split to select designs, so the decision rule was fixed and committed *before* D+B finished training (`scripts/select_final.py`, commit `9b45324`). The rule: take the lower mean **validation** $E^{(2)}$ over 1×, 2× and 3× (averaged over both seeds), keeping the default unless the difference exceeds the seed-to-seed spread. On validation, symmetrised D scores 0.02850 and symmetrised D+B 0.02862, a tie (D+B is even slightly worse) well inside the spread of 0.00093. So the final model remains **symmetrised D**. Table 5 shows why D+B cannot do better once it is wrapped. Symmetry augmentation makes the *raw* model far more consistent (C_R 0.0019 → 0.00047, the lowest of any learned model) at no cost in speed generalisation. But the wrapper already makes *any* model exactly consistent, and on the original clips D+B is no more accurate than D ($E^{(5)}$ 0.0277 against 0.0274).

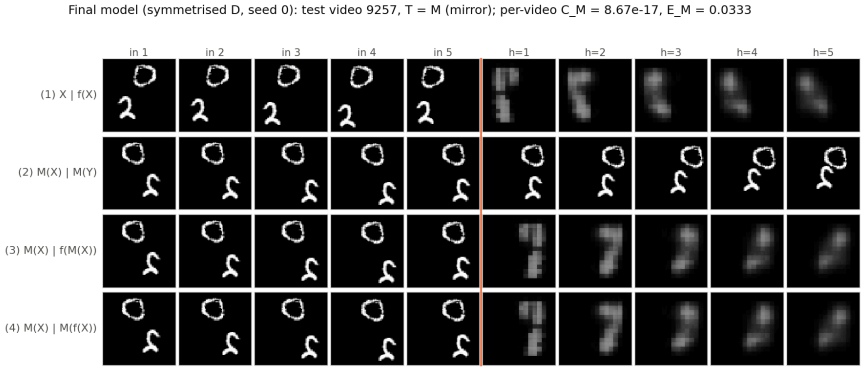
Table 6: Symmetrised models (Eq. 7), using the Table 5 columns. Training time is that of the underlying learned model (symmetrisation needs no training). Inference time covers all eight transformations, the forward passes, the inverse transformations and the averaging, for all 1,000 original test videos. C_M and C_R are at floating-point rounding level.

(a) Accuracy and consistency at 1×

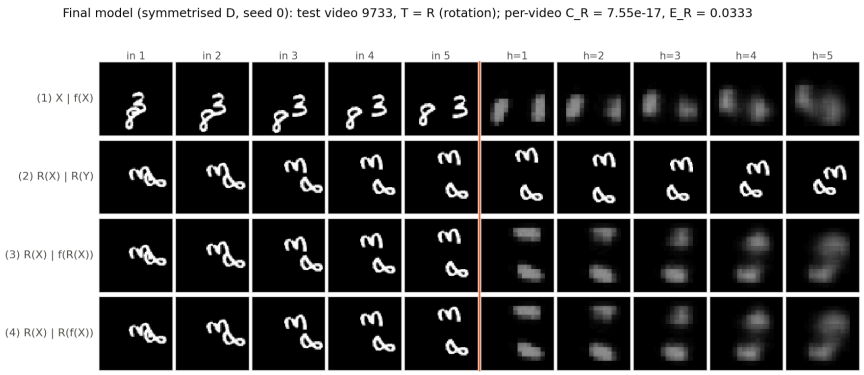
Model	$E^{(5)}$ 1×	C_M	E_M	C_R	E_R
Symmetrised B	0.0224 (0.0230, 0.0217)	7.82×10^{-17} (8.35×10^{-17}, 7.28×10^{-17})	0.0224 (0.0230, 0.0217)	6.36×10^{-17} (6.82×10^{-17}, 5.90×10^{-17})	0.0224 (0.0230, 0.0217)
Symmetrised D (final)	0.0273 (0.0269, 0.0278)	6.43×10^{-17} (6.38×10^{-17}, 6.48×10^{-17})	0.0273 (0.0269, 0.0278)	4.96×10^{-17} (5.09×10^{-17}, 4.83×10^{-17})	0.0273 (0.0269, 0.0278)
Symmetrised D+B	0.0276 (0.0273, 0.0278)	5.56×10^{-17} (5.91×10^{-17}, 5.22×10^{-17})	0.0276 (0.0273, 0.0278)	4.33×10^{-17} (4.68×10^{-17}, 3.98×10^{-17})	0.0276 (0.0273, 0.0278)

(b) Speed generalisation and cost

Model	$E^{(2)}$ 1×	$E^{(2)}$ 2×	$E^{(2)}$ 3×	Params	Train s/epoch	Inference (1,000 videos)
Symmetrised B	0.0190	0.0429	0.0492	480,784	30.1 s	15.7 s
	(0.0196, 0.0184)	(0.0430, 0.0428)	(0.0496, 0.0487)		(30.2 s, 29.9 s)	(15.7 s, 15.7 s)
Symmetrised D (final)	0.0245	0.0291	0.0329	480,784	25.8 s	16.4 s
	(0.0240, 0.0250)	(0.0285, 0.0297)	(0.0325, 0.0332)		(25.7 s, 25.8 s)	(16.3 s, 16.4 s)
Symmetrised D+B	0.0247	0.0293	0.0327	480,784	31.6 s	15.3 s
	(0.0245, 0.0249)	(0.0293, 0.0292)	(0.0327, 0.0328)		(32.2 s, 31.0 s)	(15.5 s, 15.2 s)







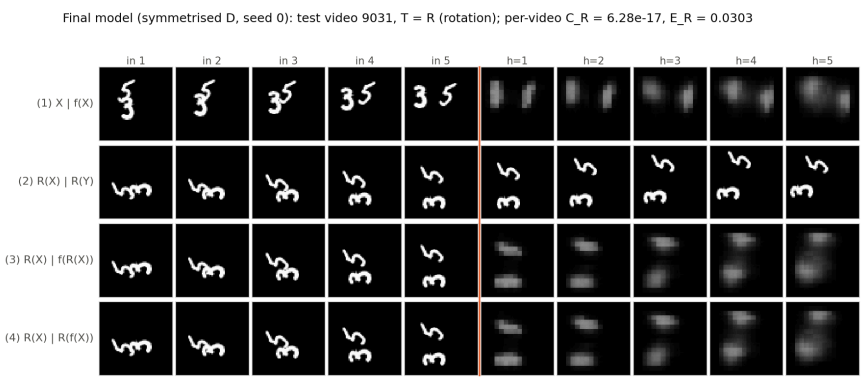


Figure 6(a): Figure 3 repeated with the final model (symmetrised D, seed 0) on exactly the same videos and transformations (9257 and 9256 with M; 9733 and 9031 with R). Rows 3 and 4 are now identical, because per-video C_T is about 10^{-16}. GIFs: `Question2/gifs/fig6a_final_M_v9257.gif`,

Question2/gifs/fig6a_final_M_v9256.gif, Question2/gifs/fig6a_final_R_v9733.gif, Question2/gifs/fig6a_final_R_v9031.gif.

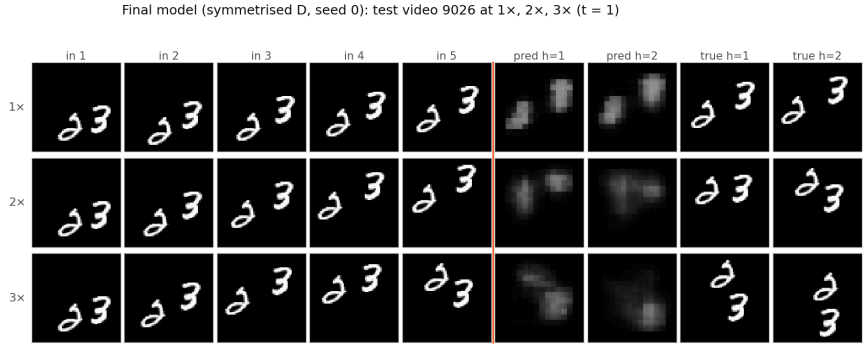


Figure 6(b): Figure 5 repeated with the final model (symmetrised D, seed 0) on the same test video (9026) at 1×, 2× and 3×.

Q2.15 — Learned versus built-in symmetry

	Learned (model B)	Built in (symmetrised B / D)
Consistency	$C_M = 0.0012$, $C_R = 0.0014$: about half of A's C_R, but clearly non-zero	C_M, $C_R \approx 5$–8×10^{-17}: zero up to rounding
Accuracy, original clips ($E^{(5)}$ 1×)	0.0230	symmetrised B 0.0224 (slightly *better* than B); symmetrised D 0.0273 (= D's 0.0274)
Accuracy, transformed clips	E_M 0.0230, E_R 0.0229	$E_M = E_R = E^{(5)}$ exactly (0.0224 / 0.0273)
Inference, 1,000 videos	0.93 s	15.7 s / 16.4 s (≈ 16–17× slower than B / D)

The optional D+B adds a third data point. Learning the symmetry on top of speed augmentation brings the raw model close to equivariance (C_M 0.00053, C_R 0.00047, and E_M, E_R within 0.1% of its $E^{(5)}$) at normal inference cost (0.99 s). It is still not exactly zero, and wrapping it (symmetrised D+B, Table 6) gives the same accuracy as symmetrised D ($E^{(5)}$ 0.0276 against 0.0273). Learned symmetry is the cheap approximation; built-in symmetry is the exact but roughly 16× more expensive guarantee.

The built-in symmetry is exact; the learned one is not. Augmentation only adds a penalty for inconsistency on the training samples it happens to draw. A network with finite capacity and 30 epochs of training approximates the symmetry, imperfectly and only near the training data. The symmetrised forecaster is equivariant *by construction*: the group-averaging argument above holds for any weights. The cost is inference. Eight forward passes on an 8× larger batch, plus the transformations, make it about 16–17× slower here (a little worse than 8× because of the transformation copies and the larger batches).

When averaging would hurt. Averaging eight predictions blurs whenever they *disagree*: if the base model is strongly tied to appearance and makes bad predictions on transformed inputs, the average mixes a sharp, correct prediction with seven misplaced ones, producing ghosts and blur. It would also hurt if the true dynamics were not symmetric (for example gravity, a preferred drift direction, or scenes where orientation carries meaning), since the average would then impose a symmetry the data do not have. **Here it did not hurt in MSE.** All three base models are already nearly equivariant (E_R within 7.5% of $E^{(5)}$ even for A), so the eight predictions agree closely and averaging mainly removes noise. $E^{(5)}$ improves slightly for B (0.0230 → 0.0224) and is unchanged for D. A supplementary check on symmetrised A (not in the tables) gives 0.0222 against 0.0223 for A. Visually, though, the averaged frames are dimmer and softer (Figure 6: peak intensity about 0.45–0.57 against 0.6–0.8). MSE rewards this hedging, while a sharpness-sensitive metric would penalise it.

Q2.16 — Back to video 9733 (rotation)

Centroids at $h = 1, 3, 5$ on the rotated clip. The truth is: "∞" rows 46.6, 46.6, 40.6; "m" rows 12.7, 20.7, 26.9.

Model (seed 0)	per-video C_R	per-video E_R	"∞" row at h=5 (true 40.6)	"m" row at h=5 (true 26.9)
A, row 3 $f(RX)$	6.74×10^{-3}	0.0366	47.6 (no bounce)	22.2 (stalls, fades to 0.48)
B (symmetry augmentation)	1.88×10^{-3}	0.0335	46.4	23.6
C (speed augmentation)	5.62×10^{-3}	0.0386	45.0	20.1
D (speed + reach)	4.65×10^{-3}	0.0371	44.6	20.4

Model (seed 0)	per-video C_R	per-video E_R	"∞" row at h=5 (true 40.6)	"m" row at h=5 (true 26.9)
Final: symmetrised D	7.55×10^{-17}	0.0333	44.6 (rows 3 = 4)	21.4

Partly. The disagreement between rows 3 and 4, which was the subject of Q2.7, is gone completely: rows 3 and 4 of Figure 6(a) are identical, and E_R on this clip is the lowest of all models (0.0333 against 0.0366 for A). **Symmetrisation fixed the inconsistency** exactly, by construction. **The missed bounce is partly repaired**: the "∞" now turns upwards (row 44.6 at $h = 5$ against 47.6 for A), but still too late and too weakly. This part comes from **speed augmentation**, not from symmetrisation. The unsymmetrised C and D already turn the "∞" (rows 45.0 and 44.6), whereas B, which saw only 1× clips, barely does (46.4). Faster training windows contain many more wall bounces, so C and D learned to reflect sooner. The larger reach adds nothing here ($C \approx D$). The **error** on this clip is lowest for the final model because averaging keeps D's bounce while removing the orientation-dependent mistakes. On their own, C and D have higher E_R than B because their 1× predictions are blurrier. **Still not right**: the "m" lags about 5 px behind the truth at $h = 5$, and both digits are dim, shapeless blobs. The extra blur comes from D being trained on three speeds (Q2.14) and from the eight-way average.

Q2.17 — Final verdict

Largely yes. It learned *where* things move, though not *what* is moving. The final model forecasts mirrored and rotated clips exactly as well as the originals ($C_T \approx 10^{-16}$, $E_M = E_R = E^{(5)}$). At 3× its error is only 34% above its 1× error (model A: +172%), and it stays far below Blank and Copy-last throughout. A sceptic should be convinced by tests that change appearance or speed but keep the motion rule: unseen shapes (letters), or speeds between and beyond 1–3× (1.5×, 4×), with error growing smoothly with displacement. One test it still fails is **object identity through overlap**: when two digits cross (videos 9007 and 9013), it merges them into one blob and never separates them again. Its long-horizon frames are blurry, not sharp.

Use of generative AI

As the course AI policy requires, this section records how a generative AI tool was used for this assignment.

Item	Details
Tool	Claude Code (Anthropic, model Claude Opus 5.5), running as a coding agent in VS Code on the author's machine, with access to the assignment folder and a terminal.
Prompt	"hello this is my PA2. it has pdf file that include all the details. please start the implementation for this assignment. I want you to first compelte the assignment based on all the requiredmetns mentioned that once everything is done. including report then we will work on understanding each and every componnent. so please proceeed. dont worry about the deadline mentioned in the pdf. my teacher has agreed to increase the deadline." (verbatim; the handout PDF was attached).
Output	Q1: the architecture diagram script (`Question1/design/architecture_diagram.py`) and the full text of the Q1 proposal. Q2: all code in `Question2/mm` and `Question2/scripts` (data loading, transforms, windows, ConvLSTM models, training, evaluation, figures, GIFs). It also trained every model, produced every table and figure, and drafted the answers to Q2.1–Q2.17. The predictions for Q2.5 and Q2.11 were written by the tool and committed to git *before* the corresponding experiments were run, as the handout requires (see the commit history). Report: the HTML/PDF build pipeline (`report/`) and this document.
Edits made by the student	[TO BE COMPLETED BY THE STUDENT: list what you changed after reviewing the output, e.g. corrections to answers, rewording, design changes in Q1, re-runs.]
Verification	The Table 1 checks, the model-A parameter count (480,784) and the exactness of the symmetrised forecaster (C_M, C_R at floating-point level) were verified programmatically. Every number in the tables is generated directly from the saved result files by `report/build.py`.